

Commenti a cursorep.c

Analizziamo il seguente codice

```
int prova[N];
```

```
int *cursore;
```

```
int valore;
```

```
for (cursore=prova+9, valore=10;
```

```
    cursore!=prova;
```

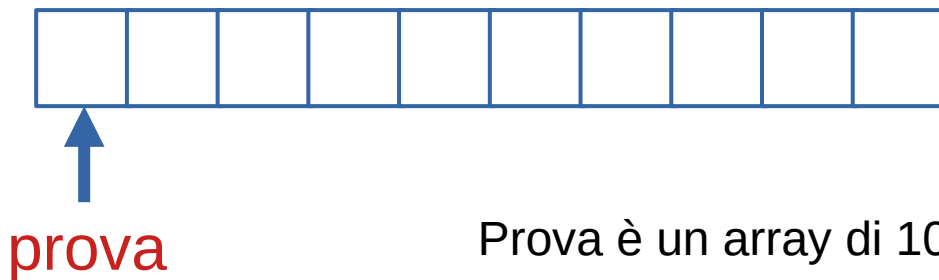
```
    cursore--,valore--)
```

```
    *cursore = valore;
```

Analizziamo il seguente codice

```
int prova[N];
```

```
int *cursore;
```



Prova è un array di 10 interi

È dichiarato staticamente

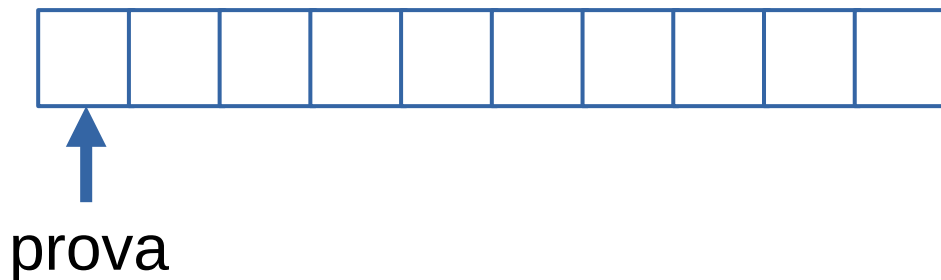
Il suo nome equivale a un puntatore costante (non può essere modificato)

Fa riferimento alla prima cella

Analizziamo il seguente codice

```
int prova[N];
```

```
int *cursore;
```



Cursore è un puntatore a intero, non inizializzato

Sia prova che cursore sono contenuti nello stack di esecuzione del processo

Analizziamo il seguente codice

```
int prova[N];
```

```
int *cursore;
```

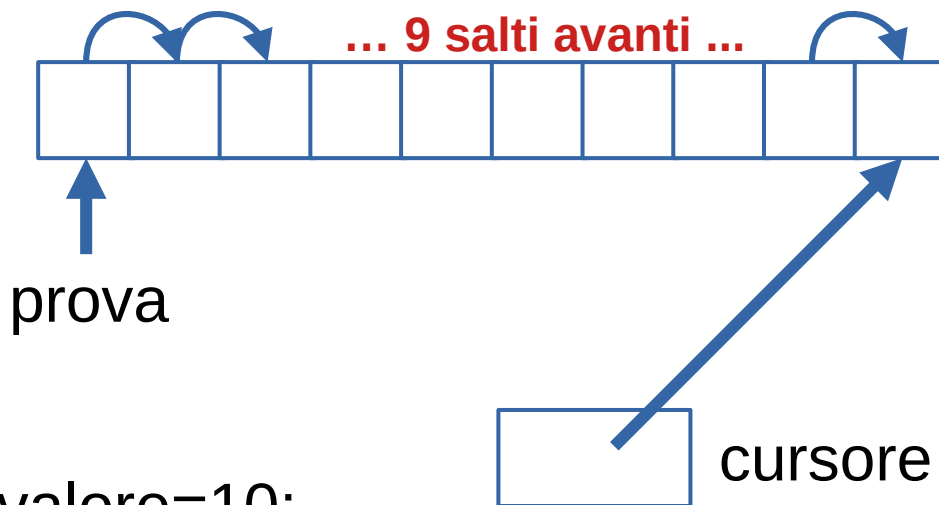
```
int valore;
```

```
for (cursore=prova+9, valore=10;
```

```
    cursore!=prova;
```

```
    cursore--,valore--)
```

```
*cursore = valore;
```



Inizializziamo cursore

Ogni +1 lo sposta avanti di una cella
(indipendentemente dal numero di
byte della stessa)

Cursore ora punta all'ultima cella dell'array

Analizziamo il seguente codice

```
int prova[N];
```

```
int *cursore;
```

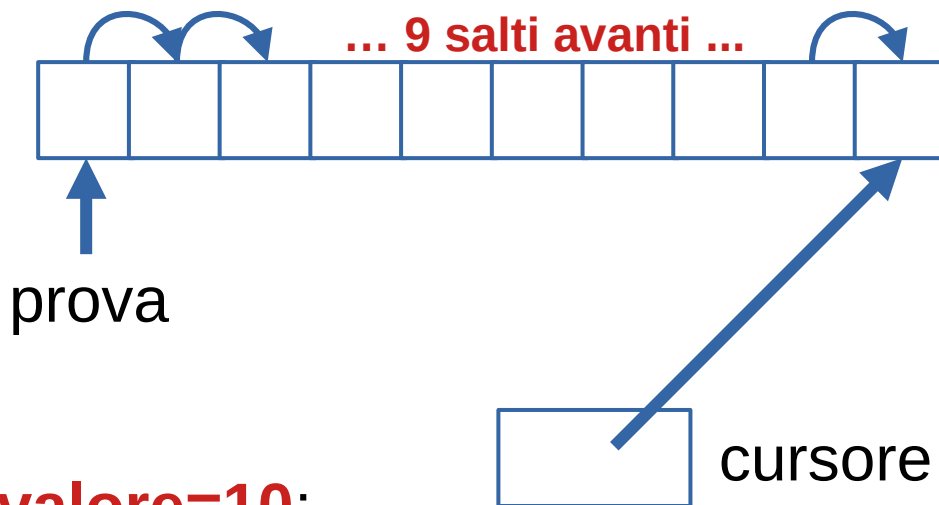
```
int valore;
```

```
for (cursore=prova+9, valore=10;
```

```
    cursore!=prova;
```

```
    cursore--,valore--)
```

```
    *cursore = valore;
```



In un for si possono inizializzare tante variabili
Basta separare le inizializzazioni con una virgola

Analizziamo il seguente codice

```
int prova[N];
```

```
int *cursore;
```

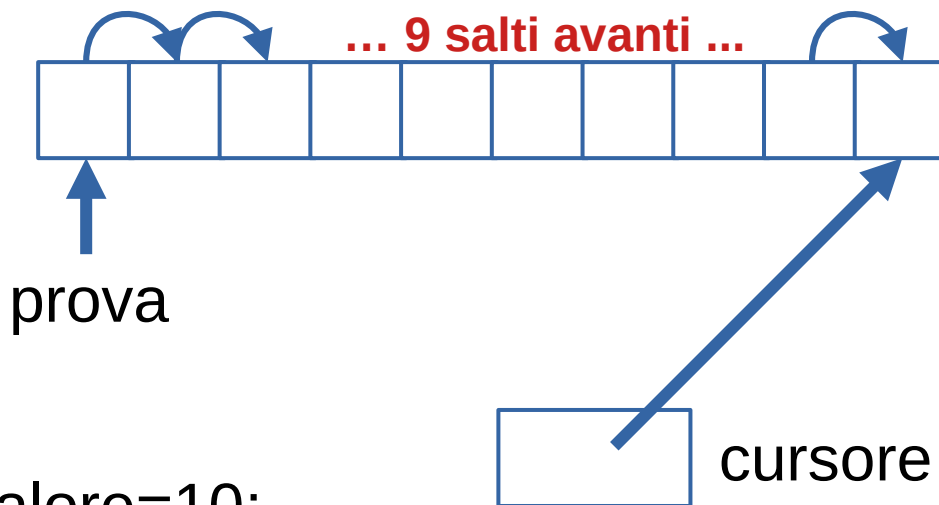
```
int valore;
```

```
for (cursore=prova+9, valore=10;
```

```
    cursore!=prova;
```

```
    cursore--,valore--)
```

```
    *cursore = valore;
```



Procederemo a ritroso

Ci fermeremo una volta raggiunta la prima cella dell'array, riferita da prova

Analizziamo il seguente codice

```
int prova[N];
```

```
int *cursore;
```

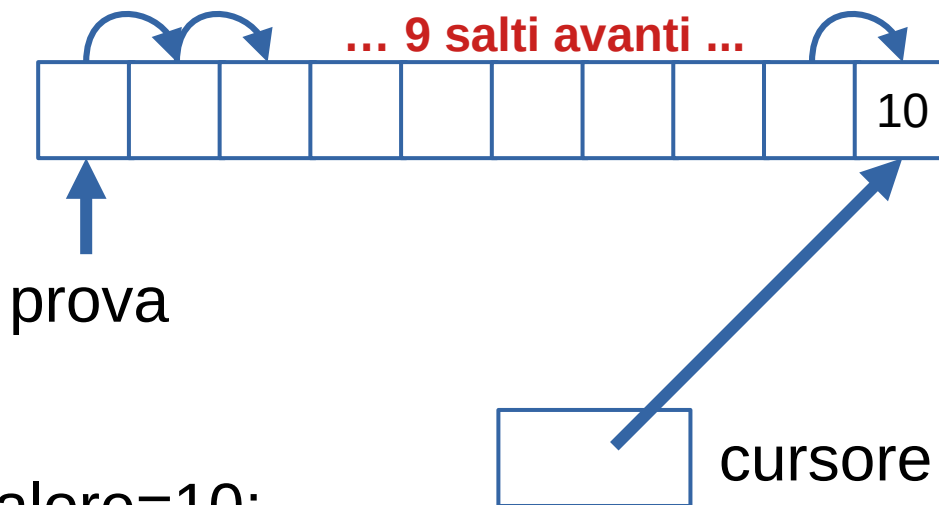
```
int valore;
```

```
for (cursore=prova+9, valore=10;
```

```
    cursore!=prova;
```

```
    cursore--,valore--)
```

```
    *cursore = valore;
```



A ogni iterazione assegneremo un valore alla cella su cui siamo posizionati

Analizziamo il seguente codice

```
int prova[N];
```

```
int *cursore;
```

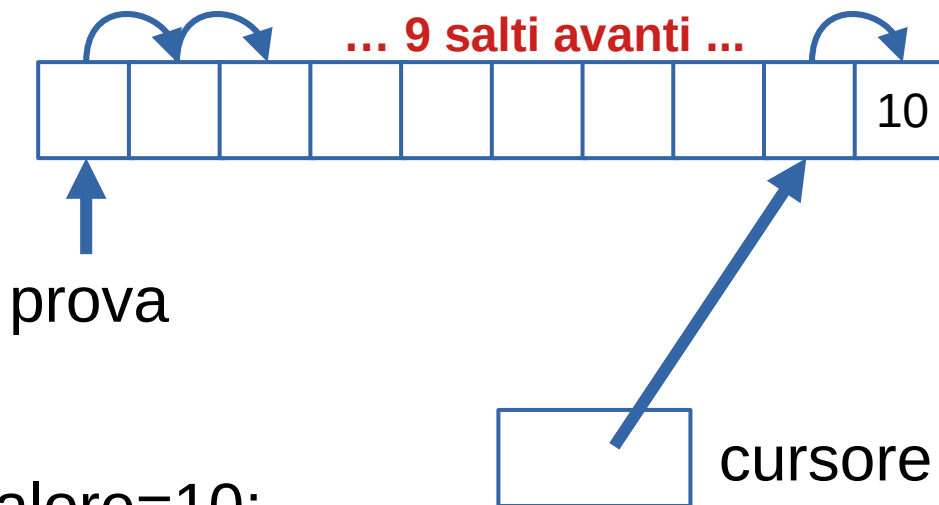
```
int valore;
```

```
for (cursore=prova+9, valore=10;
```

```
    cursore!=prova;
```

```
    cursore--,valore--)
```

```
    *cursore = valore;
```



A ogni iterazione il cursore verrà spostato

Farà riferimento alla cella precedente a quella considerata l'ultima volta

Analizziamo il seguente codice

```
int prova[N];
```

```
int *cursore;
```

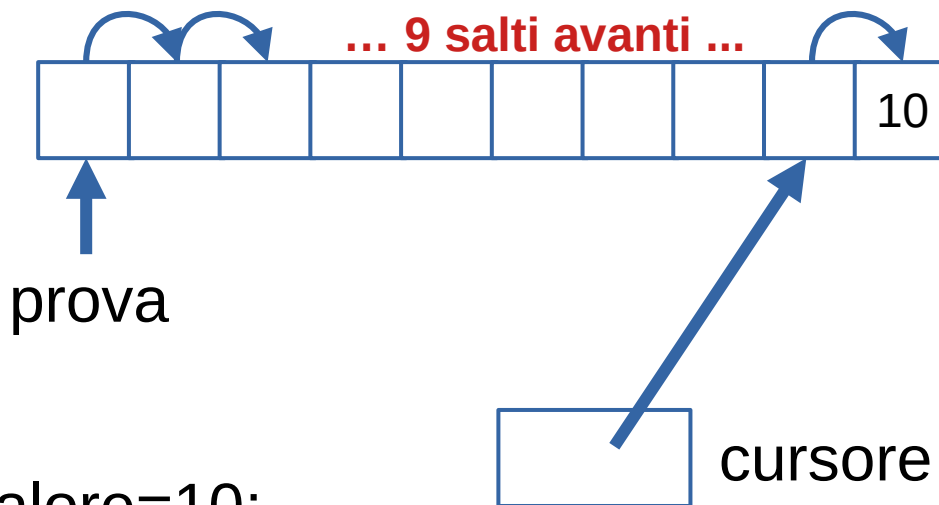
```
int valore;
```

```
for (cursore=prova+9, valore=10;
```

```
    cursore!=prova;
```

```
    cursore--, valore--)
```

```
    *cursore = valore;
```



In questo esempio anche `valore` viene modificata a ogni iterazione. Diventa via via più piccola.

In un ciclo `for` si possono fare diverse operazioni ripetute a ogni ciclo, basta separarle con la virgola

Analizziamo il seguente codice

```
int prova[N];
```

```
int *cursore;
```

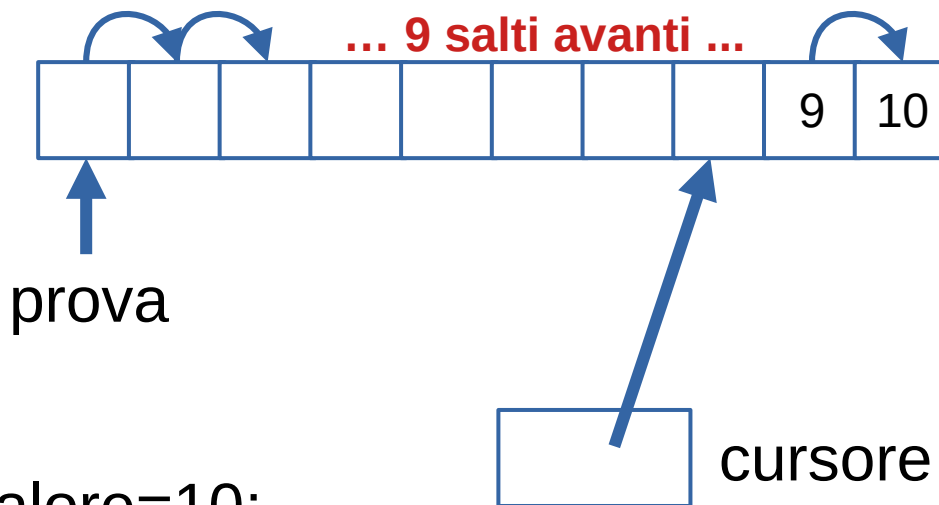
```
int valore;
```

```
for (cursore=prova+9, valore=10;
```

```
    cursore!=prova;
```

```
    cursore--,valore--)
```

```
    *cursore = valore;
```



A titolo dimostrativo andiamo indietro ancora di un passo

Analizziamo il seguente codice

```
int prova[N];
```

```
int *cursore;
```

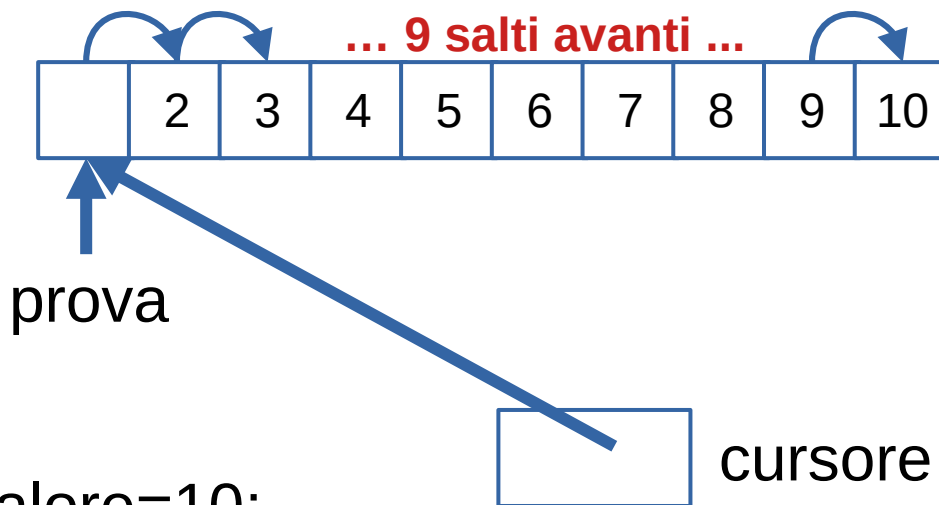
```
int valore;
```

```
for (cursore=prova+9, valore=10;
```

```
    cursore!=prova;
```

```
    cursore--,valore--)
```

```
    *cursore = valore;
```



Ci fermiamo quando cursore diventa uguale a prova

La cella di indice 0 non è stata assegnata

Nell'esempio l'assegnamento è fatto dopo il ciclo